Climbing-Hold Detection with Superpixels

[Your name and student ID]
Robot and Computer Vision — Midterm, National Taiwan Normal University

October 2026

1 Introduction and objective

The task is to find the colored holds in a picture of a climbing wall and output, for each hold, its *type*, its *position* (x, y, z) and its *orientation* (roll, pitch, yaw), where the orientation is the angle of the longest straight edge of the hold. The three superpixel algorithms SLIC, LSC and SEEDS (OpenCV `ximgproc`) have to be compared, and the choice of parameters has to be justified and its sensitivity shown.

This version of the report covers the **synthetic walls** of the perception-challenge simulator. I use three scenes (27 holds in total) generated with the simulator, each with its screenshot, the scene description (`scene.json`, used *only* to score the result), and a stereo screenshot. The real photographs are treated in Section 6. The type of a hold is its color class (red, green or blue); all holds are the same object and differ only in color.

2 Approach

The pipeline is: calibrate with the red marker → superpixels → hold candidates → type, position, yaw → depth from stereo. Nothing in the detection reads the scene file.

Calibration. The red square in the image has known size and its bottom-left corner is the origin. I segment it by color, fit its four corners and compute a homography from pixels to wall meters. A homography, not just a scale and an offset, is needed because the camera is slightly tilted: on one scene it reduced the mean position error from 2.1 cm to 0.8 cm. The handout says the marker is 1 m. This holds for walls at least 2 m wide, but the simulator draws a 0.1 m marker on narrower walls (my third scene is 0.8 m wide), so the marker size is a calibration input, read from the scene file (1 m if absent). The marker also gives the image resolution, and every image is resized to **370 px/m**, so all size parameters are the same for every wall (a hold is then about 14 px wide).

Superpixels. SLIC (the SLICO variant, Lab color, 3×3 blur), LSC and SEEDS from `cv2.ximgproc`. Each superpixel is reduced to its mean Lab color. The superpixel size is a swept parameter (Section 3); the baseline is a 5 px SLIC region.

Hold candidates. A hold is a small, compact patch whose color differs from the wall around it. For every superpixel I compute the Lab distance between its mean color and the *median* color of the superpixels in a 36 px window (median, so that the holds themselves do not bias the wall color). Superpixels with contrast above a threshold T are kept, cleaned with a morphological opening and closing, and grouped into connected components. A component is accepted if its area (29–540 px^2 at the working resolution), aspect ratio (≤ 2.5) and solidity (≥ 0.85) look like a hold, and if its saturation is at least S. The saturation floor S removes shadows and the gray faces of the volumes, which have contrast but little color. I first tried an edge-based rule (contrast with the immediate neighbors): it flagged every edge, 179 candidates for 10 holds. The median contrast removed this problem.

Type and position. The type is the nearest of the three hold colors in hue. The position (x, y) is the hold's centroid mapped through the homography.

Yaw: matching against a template. The first estimate, the longest edge of the hold's outline, was poor (mean error 41° on the holds scored in Section 4): it often selects a side of the rounded bowl instead of the flat plate with the two screw holes. Following the instructor's suggestion to match the hold against possible views, I use the silhouette: for a hold lying flat on the wall, its silhouette is a template rotated in the image plane. The

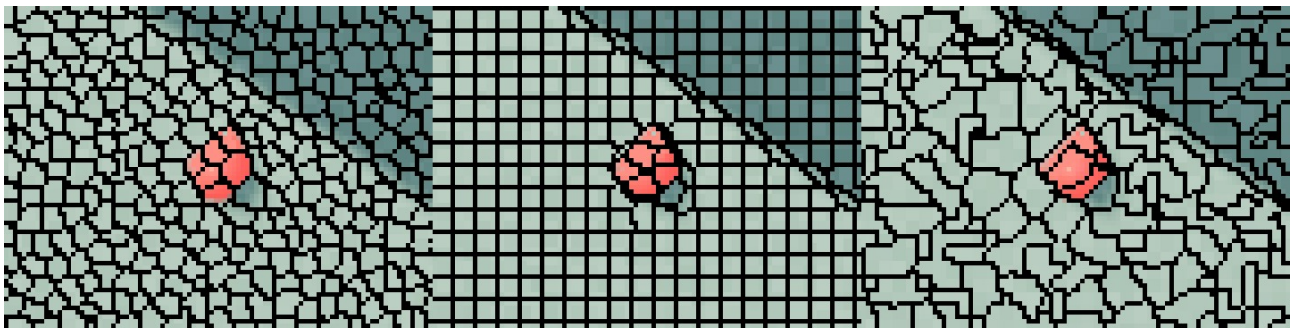


Figure 1: The same patch (a hold next to a pyramid, at the working resolution) under SLIC, LSC and SEEDS with size 5. SLIC cells are compact and follow the hold; LSC gives an almost regular grid; SEEDS gives irregular blocks.

template is the silhouette of one training hold at yaw 0 (hold 8 of the third scene; it is excluded from all yaw scores). A detected hold is cut out of the *original* screenshot, segmented by the chroma distance to the wall, normalised to the template's area and centroid, and compared with the template rotated in 3° steps (with ±3 px shifts) by intersection over union; the best angle is the yaw.

Depth z from stereo. The simulator's front stereo camera has two cameras 6.4 cm apart looking at the wall point straight ahead, with a 74° vertical field of view (taken from the simulator's source). From the height and distance of the rig I build both projection matrices. Holds are detected in each half of the stereo image, matched by color and along the epipolar line (tolerance 6 px), and triangulated with `cv2.triangulatePoints`. The expected depth error grows with the square of the distance, $\sigma_z \approx d^2 \cdot 0.5\,\mathrm{px}/(f\,b)$, so the stereo z is used only if $\sigma_z \leq 5$ cm; otherwise $z = 0$.

Implementation. `src/candidates.py` (calibration, contrast, candidates), `src/superpixels.py`, `src/orientation.py` (yaw), `src/stereo.py` (rig, triangulation), `src/evaluate.py` (scoring only); the per-wall runners are in `src/pipelines/`.

3 Parameter selection and sensitivity

Procedure. Size parameters are not tuned per wall: they follow from the working resolution. What is tuned is (i) the superpixel size, (ii) the contrast threshold T (the ROC parameter) and (iii) the saturation floor S. I swept three algorithms × seven sizes (3–14 px) × 19 thresholds (10–100) × $S \in \{0, 118\}$ on all three walls (2394 evaluations) and pooled the 27 holds. A detection counts as correct if it lies within 10 cm of an unmatched true hold (greedy matching); the score is the F1 value of the pooled true and false positives. The final setting is the one with the best pooled F1 *and no false positives*: **SLIC, 5 px, $T = 40$, $S = 118$.**

Algorithm	best size	best T	found (of 27)	false pos.	F1	time (s, 3 walls)
SLIC	5 px	40	18	0	0.80	7.9
LSC	4 px	30	17	0	0.77	25.0
SEEDS	3 px	30	18	2	0.77	4.2

Table 1: Best pooled result per algorithm (saturation floor 118). Time is for the superpixels alone at size 5 (relative only: measured with several jobs running in parallel). SEEDS snaps to a block grid, so it produces 149k superpixels where the other two produce 383k.

Findings.

- **The algorithms perform alike when the saturation floor is on** (Table 1, 17–18 of 27; the 0.03 difference in F1 is one hold and within the noise of 27 holds). SLIC was chosen because it reaches its recall without any false positive and is faster than LSC.
- **Without the saturation floor the contrast alone separates the holds worse, and the algorithms differ** (Fig. 2, left): to reach 20 of 27 holds LSC needs 4 false positives, SEEDS 7 and SLIC 20. I did not investigate why. With the floor, which rejects the gray shadows and volume faces, SLIC reaches 18 holds with no false positive, where without it 20 holds cost 20 false positives.
- **The threshold is not critical around the optimum** (Fig. 3): between $T = 30$ and 45 the result stays at 17–18 holds with 0–2 false positives; above 50 it degrades (13 holds at 55, 8 at 60) as low-contrast holds fall below T.

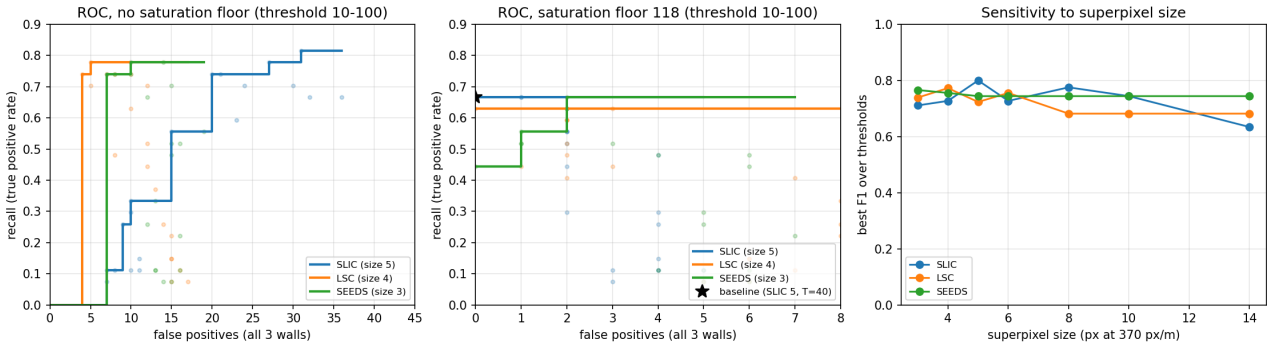


Figure 2: Left and middle: ROC curves obtained by sweeping the contrast threshold T from 10 to 100, pooled over the three walls (curves: best recall reachable with at most k false positives; faint dots: individual thresholds). Right: best F1 against the superpixel size.

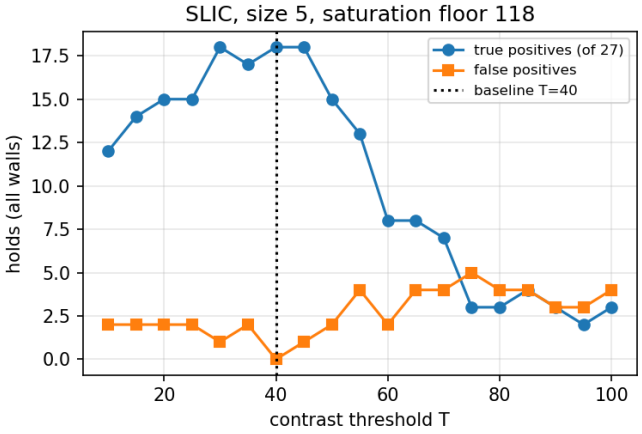


Figure 3: Sensitivity to the contrast threshold (SLIC, 5 px, saturation floor 118, all walls).

- **The superpixel size matters little** between 3 and 10 px (F1 0.68–0.80 for all three, Fig. 2, right) and degrades at 14 px, where a superpixel is as large as the hold.

4 Results

Table 2 shows the final output (type, position, orientation) of the baseline on each wall, scored against `scene.json`. Fig. 4 shows the detections.

Wall	marker	found	FP	position	type	yaw (mean / med.)	n_{yaw}
1	1 m	8 / 10	0	0.8 cm	8 / 8	15° / 3°	7
2	1 m	3 / 7	0	1.4 cm	3 / 3	55° / 69°	3
3	0.1 m	7 / 10	0	0.6 cm	7 / 7	4° / 3°	6
all		18 / 27	0	0.8 cm	18 / 18	18° / 3°	16

Table 2: Final output on the three synthetic walls. Yaw is scored modulo 180° on holds that lie flat on the wall (not on a volume), excluding the hold the template was cut from.

Type and position. All 18 detected holds have the right type, and the mean position error is 0.8 cm (maximum 2.2 cm). The error is about the size of a pixel at the working resolution (0.27 cm) times a few, and it does not depend on the wall, which supports the homography calibration.

Yaw. With template matching the median error is 3° (the step of the search), against 41° mean with the edge method, and wall 3 (holds 60–80 px wide in the original screenshot) reaches 4°. Wall 2 fails: its holds are only 6–7 px wide in the screenshot, so there is no shape to match. One hold on wall 1 is wrong by 87°: close to a 90° ambiguity, since its silhouette is nearly square at this size (`baseline/yaw_examples.png`); I have not tested this explanation further.

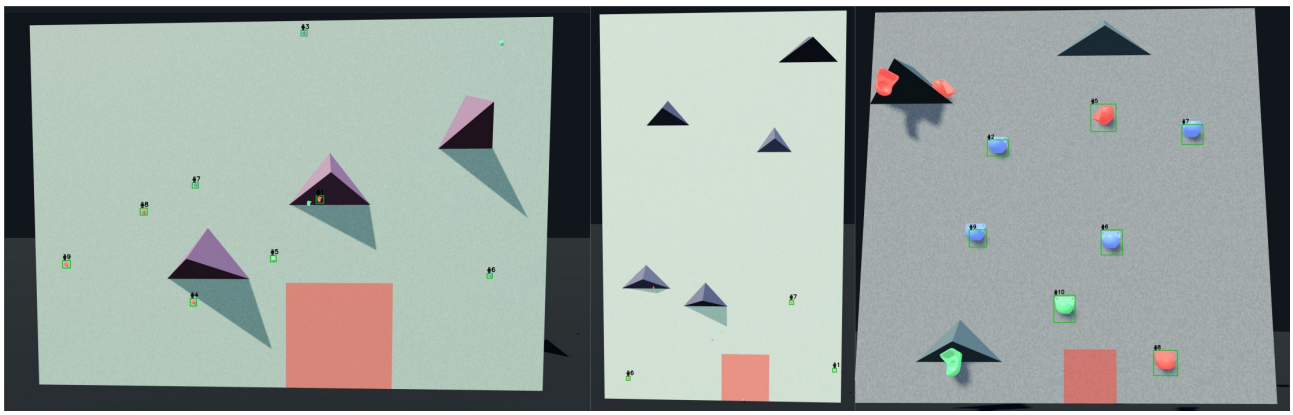


Figure 4: Detections on the three walls (green: matched to a true hold; labels give the hold number). No false positives. Missed holds are visible as uncircled holds, e.g. the red holds on and beside the dark pyramid at the top left of wall 3.

Depth. At the near camera of wall 1 (0.75 m) the stereo z has an error of about 1 cm; at 1.8–2 m (walls 2 and 3) it is 3–9 cm for holds that are flat on the wall, so the rule in Section 2 sets $z = 0$ there. The stereo view also finds few holds (2–4 per wall): each eye image is only 328 px high, so holds are only a few pixels wide.

5 Problems encountered and limits

- **Wrong marker size.** On wall 3 the first run found 0 of 10 holds, because I assumed a 1 m marker; the scene's own marker is 0.1 m. Reading the marker size and normalising the resolution fixed it. The detector thresholds were not changed.
- **Resolution.** A hold of 5–6 px (wall 2 before resizing) carries no usable information; enlarging the image 2× recovered 3 of 7 holds and removed 4 false positives, but small holds stay the weakest case.
- **Holds on volumes.** Five of the 27 holds sit on a pyramid; I find only one, and it is output with $z = 0$ and roll $= 0$, whereas the truth is $z = 0.20$ m and roll 73°. Roll and pitch are not estimated at all (always 0), which is right for the 22 holds that lie flat on the wall but wrong for these five. Over the 18 detected holds this leaves a mean roll/pitch error of 4° and a mean z error of 1.1 cm.
- **Missed holds.** Of the nine misses, four are on a volume. Of the other five, one lies under the red marker (wall 2) and one touches a dark pyramid face (wall 3); I did not diagnose the remaining three.
- **The evaluation is optimistic.** The parameters were chosen on the same three walls that are scored; the resolution handling was introduced after wall 2 and wall 3 exposed its need. The contrast threshold and saturation floor were first chosen on wall 1 and an earlier scene (since removed), so the numbers above are not a held-out test. A fresh scene should be scored with the settings frozen.
- **Template from one hold,** so the yaw estimate assumes all holds look like it; the angle convention (zero pose, sign) was inferred from the data (it matches the simulator's scene files), not from documentation.

6 Real climbing-wall photographs

This section is still to be completed. The two real photographs have no marker; the handout gives a wall grid of 20 cm, which would take the role of the red square. So far I only ran the detector in pixel space on them and inspected the result visually: large and elongated holds are missed and there are false positives at the dark top of the wall, so the synthetic parameters do not transfer unchanged and these walls need their own calibration and parameter selection.

References

[1] R. Achanta et al., "SLIC superpixels compared to state-of-the-art superpixel methods," IEEE TPAMI, 2012.

[2] Z. Li and J. Chen, "Superpixel segmentation using linear spectral clustering," CVPR, 2015.

[3] M. Van den Bergh et al., "SEEDS: superpixels extracted via energy-driven sampling," ECCV, 2012.

A Code layout and how to run

- `dataset/<wall>/`: `image.png` (or `wall.png`), `scene.json` (ground truth, synthetic walls only, used for scoring and for the marker size), `stereo.png`, `camera.json` (stereo height and distance in meters, read from the simulator page; they are not in `scene.json`).
- `src/`: `candidates.py`, `superpixels.py`, `orientation.py`, `baseline.py` (loads the baseline values), `stereo.py`, `evaluate.py`, `paths.py`; `src/pipelines/`: `run_synthetic`, `run_stereo`, `run_final` (merge into `output.json`), `experiments` (the sweep; `--replot` redraws from the saved csv), `make_template`, `make_yaw_examples`, `make_report_figures`, `run_images` and `demo_superpixels`.
- `baseline/`: everything that defines the baseline. `parameters.json` holds all tuning values (working resolution 370 px/m, hold scale 0.6, SLIC region, contrast threshold 40, saturation floor 118, size limits, stereo and yaw settings); the code reads them from there. `yaw_template.png` is the template, `yaw_template_source.png` the hold it was cut from, and `yaw_examples.png` shows the template matched to every scored hold.
- `outputs/<wall>/`: results of one wall, with its own `scripts/run.py` that holds that wall's settings. `output.json` is the final result (type, x, y, z, roll, pitch, yaw); `main_view.json` is a debug copy. The pooled sweep and the plots are in `outputs/all_walls/`.
- Run one wall: `python3 outputs/synthetic_wall1/scripts/run.py`. Optional arguments: `--marker M` (marker side, m), `--stereo-height H`, `--stereo-distance D`, `--thresh T` (default 40), `--min-sat S` (default 118), `--no-pooled`, `--force`. Every run also refreshes `outputs/all_walls/` (the sweep pooled over all walls in `dataset/`); only walls whose inputs changed are recomputed, so a new wall costs about a minute and a rerun without changes 15 s (`--force` recomputes all, `--no-pooled` skips it). A new wall: add `dataset/<wall>/` (with `scene.json`, image, optional stereo files), run `python3 src/pipelines/new_wall.py <wall>` to create its `run.py`, then run that file. The wall is scored with the baseline values in `baseline/parameters.json`, which the sweep never changes, so that score is a held-out result; but the pooled plots and tables include the wall from then on. The report builds with `pdflatex main.tex` (twice) in `report/`.
- Needs `opencv-contrib-python`, `numpy`, `matplotlib`.